Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester VI)
Artificial Intelligence Practical

Practical – I

Roll No. : T045	Name: Omkar Thukarul
Class : TYIT	Date/Time of Submission : 27/06/2026

Aim: Introduction to AI and Intelligent Agents.

1). Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

SOURCE:-

```
table = {
   ('A', 'Dirty'): 'Suck',
   ('A', 'Clean'): 'Right',
   ('B', 'Dirty'): 'Suck',
   ('B', 'Clean'): 'Right',
   ('C', 'Dirty'): 'Suck',
   ('C', 'Clean'): 'Right',
   ('D', 'Dirty'): 'Suck',
   ('D', 'Clean'): 'Left'
}
sequence_table = {
   (('A', 'Dirty'), ('A', 'Dirty')): 'Suck',
   (('A', 'Dirty'), ('A', 'Clean')): 'Right',
   (('A', 'Dirty'), ('B', 'Dirty')): 'Suck',
   (('A', 'Dirty'), ('B', 'Clean')): 'Right',
   (('A', 'Dirty'), ('C', 'Dirty')): 'Suck',
   (('A', 'Dirty'), ('C', 'Clean')): 'Right',
   (('A', 'Dirty'), ('D', 'Dirty')): 'Suck',
   (('A', 'Dirty'), ('D', 'Clean')): 'Left',
   (('A', 'Clean'), ('A', 'Dirty')): 'Suck',
   (('A', 'Clean'), ('A', 'Clean')): 'Right',
   (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
   (('A', 'Clean'), ('B', 'Clean')): 'Right',
   (('A', 'Clean'), ('C', 'Dirty')): 'Suck',
   (('A', 'Clean'), ('C', 'Clean')): 'Right',
   (('A', 'Clean'), ('D', 'Dirty')): 'Suck',
   (('A', 'Clean'), ('D', 'Clean')): 'Left',
   (('B', 'Dirty'), ('A', 'Dirty')): 'Suck',
   (('B', 'Dirty'), ('A', 'Clean')): 'Right',
   (('B', 'Dirty'), ('B', 'Dirty')): 'Suck',
```

```
    (('B', 'Dirty'), ('B', 'Clean')): 'Right',
    (('B', 'Dirty'), ('C', 'Dirty')): 'Suck',
    (('B', 'Dirty'), ('C', 'Clean')): 'Right',
    (('B', 'Dirty'), ('D', 'Dirty')): 'Suck',
    (('B', 'Dirty'), ('D', 'Clean')): 'Left',
    (('B', 'Clean'), ('A', 'Dirty')): 'Suck',
    (('B', 'Clean'), ('A', 'Clean')): 'Right',
    (('B', 'Clean'), ('B', 'Dirty')): 'Suck',
    (('B', 'Clean'), ('B', 'Clean')): 'Right',
    (('B', 'Clean'), ('C', 'Dirty')): 'Suck',
    (('B', 'Clean'), ('C', 'Clean')): 'Right',
    (('B', 'Clean'), ('D', 'Dirty')): 'Suck',
    (('B', 'Clean'), ('D', 'Clean')): 'Left',
    (('C', 'Dirty'), ('A', 'Dirty')): 'Suck',
    (('C', 'Dirty'), ('A', 'Clean')): 'Right',
    (('C', 'Dirty'), ('B', 'Dirty')): 'Suck',
    (('C', 'Dirty'), ('B', 'Clean')): 'Right',
    (('C', 'Dirty'), ('C', 'Dirty')): 'Suck',
    (('C', 'Dirty'), ('C', 'Clean')): 'Right',
    (('C', 'Dirty'), ('D', 'Dirty')): 'Suck',
    (('C', 'Dirty'), ('D', 'Clean')): 'Left',
    (('C', 'Clean'), ('A', 'Dirty')): 'Suck',
    (('C', 'Clean'), ('A', 'Clean')): 'Right',
    (('C', 'Clean'), ('B', 'Dirty')): 'Suck',
    (('C', 'Clean'), ('B', 'Clean')): 'Right',
    (('C', 'Clean'), ('C', 'Dirty')): 'Suck',
    (('C', 'Clean'), ('C', 'Clean')): 'Right',
    (('C', 'Clean'), ('D', 'Dirty')): 'Suck',
    (('C', 'Clean'), ('D', 'Clean')): 'Left',
    (('D', 'Dirty'), ('A', 'Dirty')): 'Suck',
    (('D', 'Dirty'), ('A', 'Clean')): 'Right',
    (('D', 'Dirty'), ('B', 'Dirty')): 'Suck',
    (('D', 'Dirty'), ('B', 'Clean')): 'Right',
    (('D', 'Dirty'), ('C', 'Dirty')): 'Suck',
    (('D', 'Dirty'), ('C', 'Clean')): 'Right',
    (('D', 'Dirty'), ('D', 'Dirty')): 'Suck',
    (('D', 'Dirty'), ('D', 'Clean')): 'Left',
    (('D', 'Clean'), ('A', 'Dirty')): 'Suck',
    (('D', 'Clean'), ('A', 'Clean')): 'Right',
    (('D', 'Clean'), ('B', 'Dirty')): 'Suck',
    (('D', 'Clean'), ('B', 'Clean')): 'Right',
    (('D', 'Clean'), ('C', 'Dirty')): 'Suck',
    (('D', 'Clean'), ('C', 'Clean')): 'Right',
    (('D', 'Clean'), ('D', 'Dirty')): 'Suck',
    (('D', 'Clean'), ('D', 'Clean')): 'Left'
}
percepts = []
def table_driven_agent(percept):
    percepts.append(percept)
```

```
    # Check complete percept history
    if tuple(percepts) in sequence_table:
        return sequence_table[tuple(percepts)]
    # Otherwise use simple lookup table
    return table.get(percept, "NoOp")
print("Vacuum Cleaner Table-Driven Agent (4 Rooms)")
n = int(input("Enter number of percepts: "))
for i in range(n):
    print(f"\nPercept {i+1}")
    location = input("Enter Location (A/B/C/D): ").upper()
    status = input("Enter Status (Clean/Dirty): ").capitalize()
    percept = (location, status)
    action = table_driven_agent(percept)
    print("Percept History :", percepts)
    print("Agent Action    :", action)
print("Omkar 45")
```

OUTPUT:-

```
Vacuum Cleaner Table-Driven Agent (4 Rooms)
Enter number of percepts: 4

Percept 1
Enter Location (A/B/C/D): a
Enter Status (Clean/Dirty): clean
Percept History : [('A', 'Clean')]
Agent Action    : Right

Percept 2
Enter Location (A/B/C/D): b
Enter Status (Clean/Dirty): clean
Percept History : [('A', 'Clean'), ('B', 'Clean')]
Agent Action    : Right

Percept 3
Enter Location (A/B/C/D): a
Enter Status (Clean/Dirty): dirty
Percept History : [('A', 'Clean'), ('B', 'Clean'), ('A', 'Dirty')]
Agent Action    : Suck

Percept 4
Enter Location (A/B/C/D): b
Enter Status (Clean/Dirty): dirty
Percept History : [('A', 'Clean'), ('B', 'Clean'), ('A', 'Dirty'), ('B', 'Dirty')]
Agent Action    : Suck
Omkar 45
```

2). Design a Simple Rational Agent by defining PEAS for Autonomous Taxi Environment. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

SOURCE:-

```
table = {
    ('Green', 'Clear'): 'Move',
    ('Green', 'Obstacle'): 'Brake',
    ('Yellow', 'Clear'): 'Slow',
```

```
    ('Yellow', 'Obstacle'): 'Brake',
    ('Red', 'Clear'): 'Stop',
    ('Red', 'Obstacle'): 'Stop'
}
sequence_table = {
    (('Red', 'Clear'), ('Green', 'Clear')): 'Move',
    (('Green', 'Clear'), ('Yellow', 'Clear')): 'Slow',
    (('Yellow', 'Clear'), ('Red', 'Clear')): 'Stop',
    (('Green', 'Clear'), ('Green', 'Obstacle')): 'Brake',
    (('Green', 'Obstacle'), ('Green', 'Clear')): 'Move'
}
percepts = []
def table_driven_agent(percept):
    percepts.append(percept)
    if tuple(percepts) in sequence_table:
        return sequence_table[tuple(percepts)]
    return table.get(percept, "No Action")
print("Automatic Taxi Table-Driven Agent")
n = int(input("Enter number of percepts: "))
for i in range(n):
    print(f"\nPercept {i+1}")
    signal = input("Enter Traffic Signal (Red/Yellow/Green): ").capitalize()
    road = input("Enter Road Condition (Clear/Obstacle): ").capitalize()
    percept = (signal, road)
    action = table_driven_agent(percept)
    print("Percept History :", percepts)
    print("Taxi Action     :", action)
print("Omkar 45")
```

OUTPUT:-

```
Automatic Taxi Table-Driven Agent
Enter number of percepts: 2

Percept 1
Enter Traffic Signal (Red/Yellow/Green): red
Enter Road Condition (Clear/Obstacle): clear
Percept History : [('Red', 'Clear')]
Taxi Action     : Stop

Percept 2
Enter Traffic Signal (Red/Yellow/Green): green
Enter Road Condition (Clear/Obstacle): obstacle
Percept History : [('Red', 'Clear'), ('Green', 'Obstacle')]
Taxi Action     : Brake
Omkar 45
```

3). Design a Simple Rational Agent by defining PEAS. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

SOURCE:-

```
table = {
   ('Dry', 'NoRain'): 'Water',
   ('Dry', 'Rain'): 'Wait',
   ('Wet', 'NoRain'): 'Wait',
   ('Wet', 'Rain'): 'Wait'
}

sequence_table = {
   (('Wet', 'NoRain'), ('Dry', 'NoRain')): 'Water',
   (('Dry', 'NoRain'), ('Wet', 'NoRain')): 'Wait',
   (('Wet', 'Rain'), ('Dry', 'NoRain')): 'Water',
   (('Dry', 'Rain'), ('Dry', 'NoRain')): 'Water',
   (('Dry', 'NoRain'), ('Dry', 'Rain')): 'Wait'
}

percepts = []

def table_driven_agent(percept):
   percepts.append(percept)

   if tuple(percepts) in sequence_table:
      return sequence_table[tuple(percepts)]

   return table.get(percept, "No Action")

print("Smart Irrigation Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):
   print(f"\nPercept {i+1}")

   moisture = input("Enter Soil Moisture (Dry/Wet): ").capitalize()
   rain = input("Enter Weather (Rain/NoRain): ").capitalize()
```

```
    percept = (moisture, rain)

    action = table_driven_agent(percept)

    print("Percept History :", percepts)
    print("Agent Action    :", action)

print("Omkar 45")
```

OUTPUT:-

```
Smart Irrigation Table-Driven Agent
Enter number of percepts: 2

Percept 1
Enter Soil Moisture (Dry/Wet): dry
Enter Weather (Rain/NoRain): rain
Percept History : [('Dry', 'Rain')]
Agent Action    : Wait

Percept 2
Enter Soil Moisture (Dry/Wet): wet
Enter Weather (Rain/NoRain): norain
Percept History : [('Dry', 'Rain'), ('Wet', 'Norain')]
Agent Action    : No Action
Omkar 45
```